VDR Arithmetic: Value, Decimal, Remainder

Exact Finite Arithmetic in Irreducible Triple Form

Registry: [HOWL-VDR-1-2026]

DOI: 10.5281/zenodo.zzz

Date: May 2026

Domain: Applied Philosophy

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

Floating-point and decimal arithmetic systems lose exact equality under repeated operations. This loss is structural — it comes from representing values as single scalars that discard intermediate structure at every step. This paper introduces VDR, an arithmetic system that represents every value as a finite tree of integer triples `[V, D, R]` where V is the value slot, D is the denominator frame, and R is the remainder — exact unresolved structure that scalar systems would discard. The remainder is not error. It is part of the value.

The system provides exact rational arithmetic with zero drift over arbitrary operation chains, exact matrix inversion of ill-conditioned matrices where floating-point fails, recursive construction of irrational values where every expansion step is itself exact, and discrete calculus operators where every derivative and integral is an exact rational at every step size. A working Python implementation accompanies this paper. Every claim is verified by executable tests. The code is the specification.

---

1. The Problem: Equality Lost

Consider a simple test. Take a value, add something to it, then subtract the same thing. The result should be the original value. In exact arithmetic, it always is. In floating-point, it is not.

```
x = 1/7

delta = 1/13

x = x + delta
```

```
x = x - delta

print(x == 1/7)    # False

print(x - 1/7)     # 2.7755575615628914e-17
```

The error is small but real. It is not a bug in the implementation. It is a structural property of the representation. Floating-point encodes values as finite binary fractions with bounded mantissa width. Every operation that produces a result outside that width truncates. The truncation is silent. It accumulates. After enough operations, the original value is unrecoverable.

This is not a hypothetical concern. Numerical drift in long computation chains is a known source of failure in scientific computation, financial calculation, and geometric algorithms. The standard response is tolerance — accepting that exact equality is unavailable and replacing it with approximate comparison within epsilon bounds. This works in practice. But it means that the system has permanently lost the ability to know whether two values are the same.

VDR is a system that does not lose this ability.

```
from vdr import VDR

x = VDR(1, 7)

delta = VDR(1, 13)

x = x + delta

x = x - delta

print(x == VDR(1, 7))    # True
```

Not approximately true. Exactly true. The values are structurally identical after normalization. No epsilon. No tolerance. No accumulated error.

This holds not just for two operations but for any number. The same test with 100 additions followed by 100 subtractions:

```
x = VDR(1, 7)

step = VDR(1, 13)

for _ in range(100):

x = x + step
```

```
for _ in range(100):

x = x - step

print(x == VDR(1, 7))    # True
```

Two hundred operations. Zero drift. Exact recovery.

This paper presents the system that makes this possible. It introduces the representation, the arithmetic, the operations, and the concrete demonstrations. It does not claim that VDR replaces the real numbers or makes floating-point obsolete. It claims that exact equality can be preserved across arbitrary chains of rational computation, and that this preservation has practical consequences worth examining.

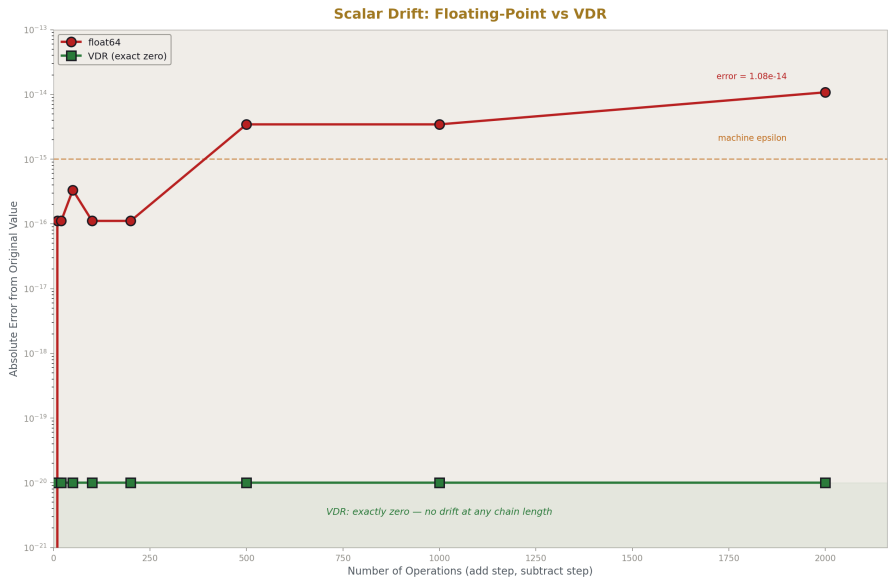

Figure 1: Fig. 1: Scalar Drift — Float error accumulates over operations while VDR stays at exact zero.

2. The Triple

A VDR object is an ordered triple of the form:

$$[V, D, R]$$

where:

- V is the **value slot**, an arbitrary-precision integer. It carries the settled numerator component in the current denominator frame.
- D is the **denominator slot**, a nonzero arbitrary-precision integer. It defines the frame in which V and R are interpreted.
- R is the **remainder slot**. It carries exact structure that the V/D frame could not absorb.

The remainder takes one of two concrete forms. An **atomic remainder** is a single integer r. A **composite remainder** is an integer base r plus a finite ordered list of child VDR triples:

$$R = r + X_1 + X_2 + \cdots + X_n$$

where each X_i is itself a valid VDR triple and n is finite.

Recursion exists only in R. Neither V nor D may contain nested VDR objects. The tree always branches through the third slot and only through the third slot. Every valid VDR object is a finite rooted tree with finite depth, finite branching at every node, and finite total node count.

In Python:

```
from vdr import VDR, Remainder

a = VDR(1, 2)                          # [1, 2, 0] — one-half

b = VDR(5)                             # [5, 1, 0] — the integer 5

c = VDR(2, 5, 1)                       # [2, 5, 1] — active, carries remainder

d = VDR(1, 3, Remainder(0, [VDR(1, 6)]))   # [1, 3, [1, 6, 0]] — nested chi
```

The triple `[1, 2, 0]` has V = 1, D = 2, and R = 0. The remainder is zero, meaning the value is fully settled in the denominator-2 frame. The triple `[2, 5, 1]` has V = 2, D = 5, and R = 1. The remainder is nonzero — this object carries exact state beyond what the denominator-5 frame absorbed. The triple `[1, 3, [1, 6, 0]]` has V = 1, D = 3, and R = a child triple `[1, 6, 0]`. The remainder contains a nested VDR object carrying structure in its own denominator-6 frame.

The remainder is not error. It is not rounding residue. It is not dispensable annotation. It is exact value-bearing structure that is part of the object. A nonzero remainder means the object carries unresolved exact state. This state is preserved through all operations. No valid operation may discard it.

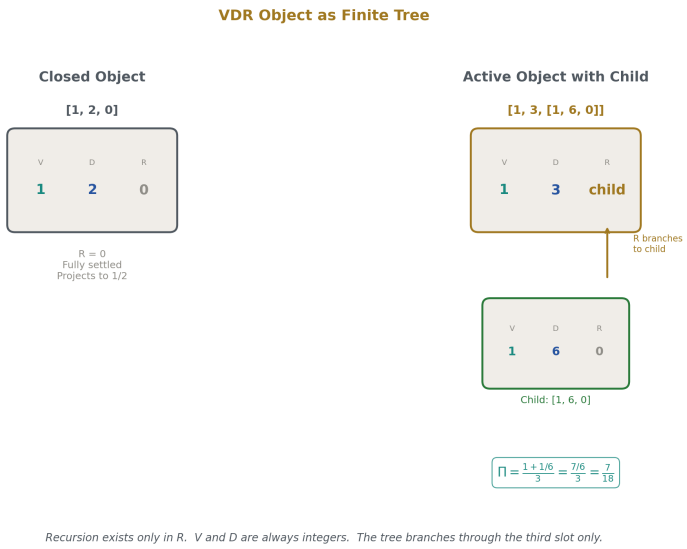

Figure 2: Fig. 6: VDR Tree Structure — Closed object [1,2,0] vs active object [1,3,[1,6,0]] showing recursion through R only.

---

3. Closed Objects and Rational Arithmetic

A VDR object with remainder zero is called **closed**:

$$[V, D, 0]$$

A closed object behaves exactly like a rational number. Its scalar projection is:

$$\Pi([V, D, 0]) = \frac{V}{D}$$

The four arithmetic operations on closed objects are defined by the standard rational formulas:

Addition:

$$[V_1, D_1, 0] + [V_2, D_2, 0] = [V_1 D_2 + V_2 D_1,\ D_1 D_2,\ 0]$$

Subtraction:

$$[V_1, D_1, 0] - [V_2, D_2, 0] = [V_1 D_2 - V_2 D_1,\ D_1 D_2,\ 0]$$

Multiplication:

$$[V_1, D_1, 0] \times [V_2, D_2, 0] = [V_1 V_2,\ D_1 D_2,\ 0]$$

Division ($V_2 \neq 0$):

$$[V_1, D_1, 0] \div [V_2, D_2, 0] = [V_1 D_2,\ D_1 V_2,\ 0]$$

All results are subject to normalization (Section 5). Division by $[0, D, 0]$ is invalid and fails explicitly.

```
a = VDR(1, 2)

b = VDR(1, 3)

print(a + b)     # [5, 6, 0]      — 1/2 + 1/3 = 5/6

print(a - b)     # [1, 6, 0]      — 1/2 - 1/3 = 1/6

print(a * b)     # [1, 6, 0]      — 1/2  $\times$  1/3 = 1/6

print(a / b)     # [3, 2, 0]      — 1/2 ÷ 1/3 = 3/2
```

The closed subclass is arithmetically closed under all four operations (excluding division by zero). It satisfies commutativity, associativity, and distributivity under value equality. The additive identity is $[0, 1, 0]$. The multiplicative identity is $[1, 1, 0]$. The additive inverse of $[V, D, 0]$ is $[-V, D, 0]$.

Under projection, closed VDR arithmetic agrees exactly with Python's `fractions.Fraction` arithmetic. This is the verifiable rational core of the system. It can be tested against any reference rational implementation and will produce identical results.

The practical consequence is exact equality recovery. Start with any closed VDR value. Perform any sequence of closed arithmetic operations. Reverse the operations. The result is exactly the original value.

```
x = VDR(3, 7)

# add then subtract

y = x + VDR(5, 13)
```

```
z = y - VDR(5, 13)

assert z == x     # True — exact recovery

# multiply then divide

y = x * VDR(11, 3)

z = y / VDR(11, 3)

assert z == x     # True — exact recovery
```

This holds for any chain length. The 200-operation test in Section 1 is not a special case. It is the general behavior of the system.

---

4. Active Objects and the Remainder

Not every value can be expressed as a closed triple in every denominator frame. The rational $1/2$ is exactly $[1, 2, 0]$ in the denominator-2 frame. But what if we need to express it in the denominator-3 frame?

Closed rebasing from denominator 2 to denominator 3 requires computing $1 \times 3/2 = 3/2$, which is not an integer. There is no integer V such that $[V, 3, 0]$ projects to $1/2$. Closed rebasing fails.

A scalar system would either refuse the operation or approximate. VDR does neither. Instead, it constructs an **active** object:

$$[1, 3, [1, 2, 0]]$$

This object has V = 1, D = 3, and R = the child triple $[1, 2, 0]$. The remainder carries the exact part of $1/2$ that the denominator-3 frame could not absorb.

The native reading is: "one-third, with exact completion $[1, 2, 0]$." The remainder completes the value. It is not added to $1/3$ in the ordinary sense. It is the exact structure that finishes what the $1/3$ frame started.

For external comparison, the value can be projected through legacy conversion:

$$\Pi([1, 3, [1, 2, 0]]) = \frac{1 + \Pi([1, 2, 0])}{3} = \frac{1 + 1/2}{3} = \frac{3/2}{3} = \frac{1}{2}$$

The projection recovers the original value exactly. The remainder $[1, 2, 0]$ contributes within the parent's denominator frame — it is divided by 3, not added externally. This

is **denominator-sensitive completion**: the remainder is interpreted in the context of the parent denominator.

```
x = VDR(1, 2)

r = x.rebase(3)

print(r)                    # [1, 3, [1, 2, 0]]

print(r.to_fraction())      # 1/2 — exact projection
```

An important semantic commitment follows from this design. The object $[2, 5, 1]$ is not the same as $[3, 5, 0]$, even though both project to $3/5$ under legacy conversion:

```
a = VDR(2, 5, 1)      # active — V=2, D=5, R=1

b = VDR(3, 5)          # closed — V=3, D=5, R=0

print(a == b)          # False
```

Both project to $3/5$. But they are different objects. The first carries remainder state 1. The second carries no remainder. The remainder is part of the object's identity, not a pending simplification.

This is the key distinction between VDR and ordinary rational arithmetic. A rational number is a single scalar. A VDR object is a triple with exact structural state. Two objects that happen to project to the same scalar may still be distinct as native VDR objects because they carry different remainder structure.

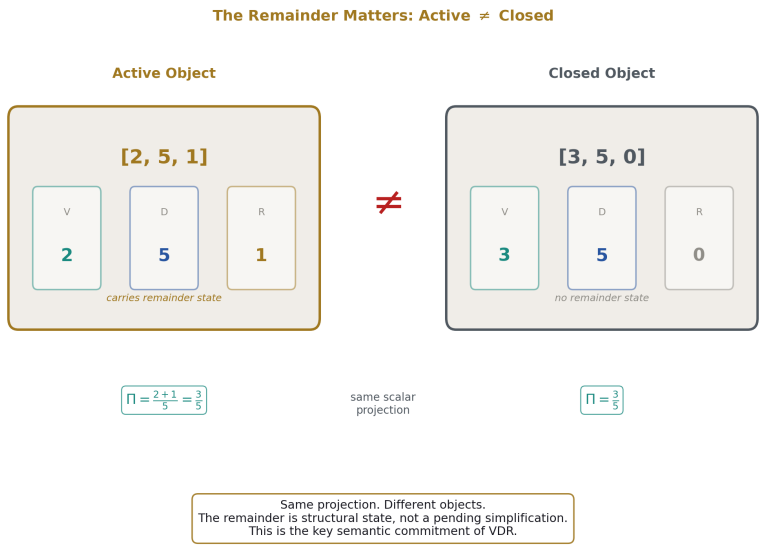

Figure 3: Fig. 8: Active vs Closed — [2,5,1] and [3,5,0] project to the same scalar but are different VDR objects.

---

5. Equality and Normalization

VDR maintains two equality relations.

Structural equality ($\equiv_s$): Two objects are structurally equal when every slot matches exactly, recursively and in order. This is identity of representation.

$$[V_1, D_1, R_1] \equiv_s [V_2, D_2, R_2] \iff V_1 = V_2 \wedge D_1 = D_2 \wedge R_1 \equiv_s R_2$$

Normalized value equality ($\equiv_n$): Two objects are value-equal when their normalized forms are structurally equal.

$$X \equiv_n Y \iff \text{norm}(X) \equiv_s \text{norm}(Y)$$

These relations are not the same. The objects $[2, 4, 0]$ and $[1, 2, 0]$ are structurally distinct but value-equal, because normalization reduces the first to the second:

```
a = VDR(2, 4)

b = VDR(1, 2)
```

```
print(a.structural_eq(b))    # False

print(a == b)                # True (value equality)
```

Normalization is the procedure that transforms a valid VDR object into a canonical form without changing its value. The rules, applied recursively bottom-up:

1. **Sign convention.** The denominator is made positive. If $D < 0$, negate both V and D. So $[1, -2, 0]$ normalizes to $[-1, 2, 0]$.
2. **GCD reduction.** For closed nodes, divide both V and D by $\gcd(|V|, |D|)$. So $[6, 15, 0]$ normalizes to $[2, 5, 0]$.
3. **Atomic remainder consolidation.** All integer contributions at the same remainder level are combined into a single base.
4. **Child normalization.** Every child is normalized before the parent.
5. **Canonical child ordering.** Remainder children are sorted by denominator magnitude, then value slot, then remainder structure.
6. **Same-denominator child merge.** Closed children sharing a denominator are added together. If the sum is zero, the child is removed.
7. **Closed-form preference.** If the entire remainder normalizes to zero, the object settles to closed form.

Normalization is deterministic, finite, and idempotent: $\text{norm}(\text{norm}(X)) = \text{norm}(X)$.

Python's == operator uses value equality. Hash values are computed from normalized form, so VDR objects work correctly as dictionary keys and set members.

---

6. Lift and Rebase

When the denominator frame of a VDR object changes, the remainder must be transported into the new frame. Two operations handle this.

6.1 Lift

Lift is the remainder transport operator. When a parent's denominator frame is scaled by integer factor k, lift rewrites the remainder so it contributes correctly in the new frame.

For atomic remainder:

$$\text{lift}(r, k) = kr$$

For composite remainder:

$$\text{lift}(r + X_1 + \cdots + X_n,\ k) = kr + \text{lift}(X_1, k) + \cdots + \text{lift}(X_n, k)$$

For a child VDR triple:

$$\text{lift}([V, D, R],\ k) = [kV,\ D,\ \text{lift}(R, k)]$$

The critical design decision: lift scales V and R of a child but does not touch the child's D. The child keeps its own internal denominator identity. The numerator and remainder absorb the outer frame scaling. This is because lift is reweighting the child's contribution into a new parent frame, not restructuring the child internally.

Lift is identity at $k = 1$, negation at $k = -1$, and composes multiplicatively: $\text{lift}(\text{lift}(R, a), b) = \text{lift}(R, ab)$. It distributes over remainder addition, preserves zero, preserves validity, and terminates in finite time.

```
r = Remainder(1)

print(r.lift(3))          # 3

child = VDR(1, 3)

lifted = child._lift_vdr(2)

print(lifted)             # [2, 3, 0] — V scaled, D preserved
```

6.2 Rebase

Rebase changes the top-level denominator of a VDR object while preserving exact value.

Closed rebase. For $[V, D, 0]$ and target denominator B, closed rebase succeeds when VB/D is an integer:

$$\text{rebase}([V, D, 0],\ B) = \left[\frac{VB}{D},\ B,\ 0\right]$$

If VB/D is not an integer, closed rebase fails and active rebase is used instead.

Active rebase. The construction proceeds in steps:

1. Compute the scaled numerator demand: $N = V \cdot B$
2. Integer divide by the source denominator: $N = Q \cdot D + S$
3. The mismatch witness $[S, D, 0]$ captures the exact part that denominator B could not absorb
4. The existing remainder is lifted into the new frame: $\text{lift}(R, B)$

5. The rebased form is: $[Q,\ B,\ [S, D, 0] + \text{lift}(R, B)]$

The projection check confirms correctness. For a closed source $[V, D, 0]$ rebased to $[Q, B, [S, D, 0]]$:

$$\Pi([Q, B, [S, D, 0]]) = \frac{Q + S/D}{B} = \frac{QD + S}{BD} = \frac{VB}{BD} = \frac{V}{D}$$

Same-denominator rebase is identity. Rebase preserves value equality, not structural equality. It is deterministic, finite, and exact.

```
x = VDR(1, 2)

print(x.rebase(4))     # [2, 4, 0]          — closed rebase

print(x.rebase(3))     # [1, 3, [1, 2, 0]] — active rebase

print(x.rebase(3).to_fraction())  # 1/2  — value preserved
```

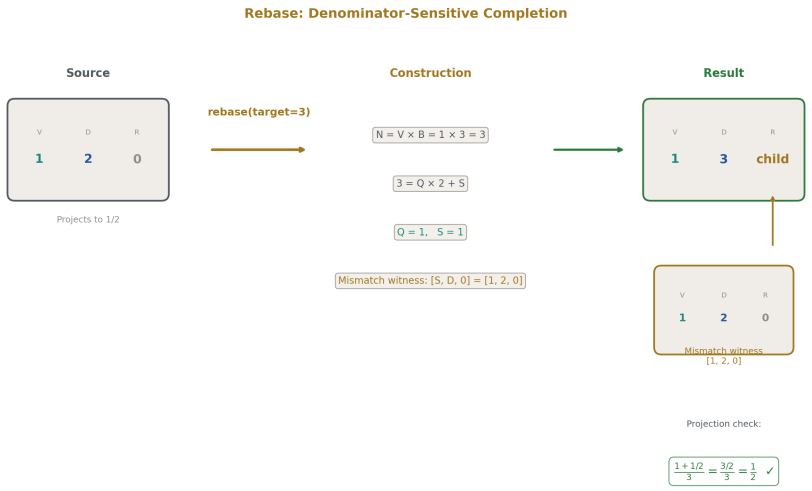

Figure 4: Fig. 7: Rebase Construction — [1,2,0] rebased to denominator 3 produces [1,3,[1,2,0]] with exact projection check.

---

7. Active Arithmetic

Arithmetic extends beyond the closed subclass to handle objects with nonzero remainders.

7.1 Same-Denominator Operations

When two VDR objects share a denominator, addition and subtraction are direct:

$$[V_1, D, R_1] + [V_2, D, R_2] = [V_1 + V_2,\ D,\ R_1 \oplus R_2]$$

where $\oplus$ combines remainder structures: atomic bases are summed, child lists are concatenated, then normalization is applied.

```
p = VDR(2, 5, 1)

q = VDR(3, 5, -1)

print(p + q)        # [1, 1, 0] — remainders cancel, result closes
```

7.2 Different-Denominator Operations

When denominators differ, both operands are cross-scaled to a shared frame $D_1 \cdot D_2$ with remainder transport via lift:

$$[V_1, D_1, R_1] + [V_2, D_2, R_2] = [V_1 D_2 + V_2 D_1,\ D_1 D_2,\ \text{lift}(R_1, D_2) + \text{lift}(R_2, D_1)]$$

7.3 Active Multiplication

For $[V_1, D_1, R_1] \times [V_2, D_2, R_2]$, the product is constructed in denominator frame $D_1 \cdot D_2$ with closed numerator $V_1 \cdot V_2$. The remainder captures three cross-term contributions:

1. V_1 scales the second operand's remainder
2. V_2 scales the first operand's remainder
3. Both remainders interact through their projected values

Scaling a remainder by integer k multiplies its base by k and scales each child's value slot by k. The remainder-times-remainder cross-product is computed by projecting both remainders to exact rationals, multiplying, and expressing the result as VDR remainder structure.

Commutativity, associativity, and distributivity hold under projection across all combinations of active and closed operands:

```
a = VDR(1, 2, 1)

b = VDR(2, 3, -1)

c = VDR(3, 5, 2)
```

```
# associativity
assert ((a * b) * c).to_fraction() == (a * (b * c)).to_fraction()
# distributivity
assert (a * (b + c)).to_fraction() == (a * b + a * c).to_fraction()
```

7.4 Active Division

Division by a closed object is multiplication by its reciprocal. Division by an active object projects the divisor to an exact rational, inverts it, and multiplies. The projected value is exact — no precision is lost. But the divisor's structural remainder information is not preserved in the result. This is a v1 compromise, documented honestly: division by active objects goes through the projection boundary.

Division by any object projecting to zero raises an explicit error.

```
x = VDR(3, 7, 2)
y = x * VDR(5, 11, -1)
z = y / VDR(5, 11, -1)
assert z.to_fraction() == x.to_fraction()   # exact roundtrip
```

7.5 Negation

Negation is recursive through the remainder:

$$-[V, D, R] = [-V, D, -R]$$

where remainder negation negates the atomic base and all children:

$$-(r + X_1 + \cdots + X_n) = -r + (-X_1) + \cdots + (-X_n)$$

---

8. Linear Algebra

VDR provides exact rational linear algebra through vector and matrix containers built on VDR arithmetic.

8.1 Vectors and Matrices

A VDR vector is an ordered list of VDR objects. A VDR matrix is a list of row vectors of equal dimension.

```
from vdr import Vec, Mat

v = Vec([VDR(1, 2), VDR(1, 3)])

m = Mat.from_ints([[1, 2], [3, 4]])

I = Mat.identity(2)
```

Vectors support addition, subtraction, scalar multiplication, dot product, and negation. Matrices support addition, subtraction, scalar multiplication, matrix multiplication, transpose, and matrix-vector product. All operations are exact VDR arithmetic.

8.2 Determinant, Inverse, Solve, Rank

Determinant is computed by cofactor expansion. For a 2×2 matrix:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

For larger matrices, the first-row cofactor expansion is used recursively. Every multiplication and subtraction is exact VDR arithmetic.

Inverse is computed by the adjugate method: $A^{-1} = \text{adj}(A) / \det(A)$. Every element of the cofactor matrix is an exact VDR determinant. The final division is exact VDR division. The inverse fails explicitly if the determinant is zero.

Solve uses Cramer's rule: for $Ax = b$, each component x_j is the determinant of A with column j replaced by b, divided by $\det(A)$. Exact throughout.

Rank is computed by Gaussian elimination with exact VDR pivot operations.

8.3 Demonstration: The Hilbert Matrix

The Hilbert matrix H is defined by $H_{ij} = 1/(i + j + 1)$ for zero-indexed i, j. It is notoriously ill-conditioned. The condition number of the $n \times n$ Hilbert matrix grows exponentially. By $n = 4$, floating-point inversion produces results contaminated by rounding error. By $n = 12$, standard double-precision float inversion is effectively useless.

In VDR, the Hilbert matrix is constructed from exact VDR rationals:

```
n = 4

H = Mat([[VDR(1, i + j + 1) for j in range(n)] for i in range(n)])
```

The 4×4 Hilbert matrix:

$$H = \begin{pmatrix} 1 & 1/2 & 1/3 & 1/4 \\ 1/2 & 1/3 & 1/4 & 1/5 \\ 1/3 & 1/4 & 1/5 & 1/6 \\ 1/4 & 1/5 & 1/6 & 1/7 \end{pmatrix}$$

Its determinant is $1/6048000$, an exact VDR rational. Its inverse is computed exactly. Every element of H^{-1} is an exact integer — the inverse of the Hilbert matrix has integer entries, a known fact that VDR recovers automatically:

```
Hi = H.inv()

product = H * Hi

assert product == Mat.identity(4)    # True — every element exactly correct
```

The product $H \times H^{-1}$ is exactly the identity matrix. Not approximately. Every diagonal element is exactly $[1, 1, 0]$. Every off-diagonal element is exactly $[0, 1, 0]$. No tolerance required.

The double inverse also recovers exactly:

```
Hi2 = Hi.inv()

assert Hi2 == H    # True — inv(inv(H)) = H exactly
```

And a long-chain stress test: multiply by an arbitrary invertible matrix B, then by B^{-1}, twenty times:

```
B = Mat([[VDR(7, 3), VDR(2, 5)], [VDR(1, 9), VDR(4, 7)]])

Bi = B.inv()

current = A

for _ in range(20):

current = current * B

current = current * Bi

assert current == A    # True — 40 matrix operations, zero drift
```

Forty matrix operations. Zero drift. Exact recovery. No floating-point system can match this on the Hilbert matrix or on the long-chain stress test.

For comparison, the same 4×4 Hilbert matrix inverted in standard double-precision floating-point produces $H \times H^{-1}$ with off-diagonal residuals on the order of 10^{-13}. These residuals are small but nonzero, and they grow with matrix size and operation count. In VDR, they are exactly zero, always.

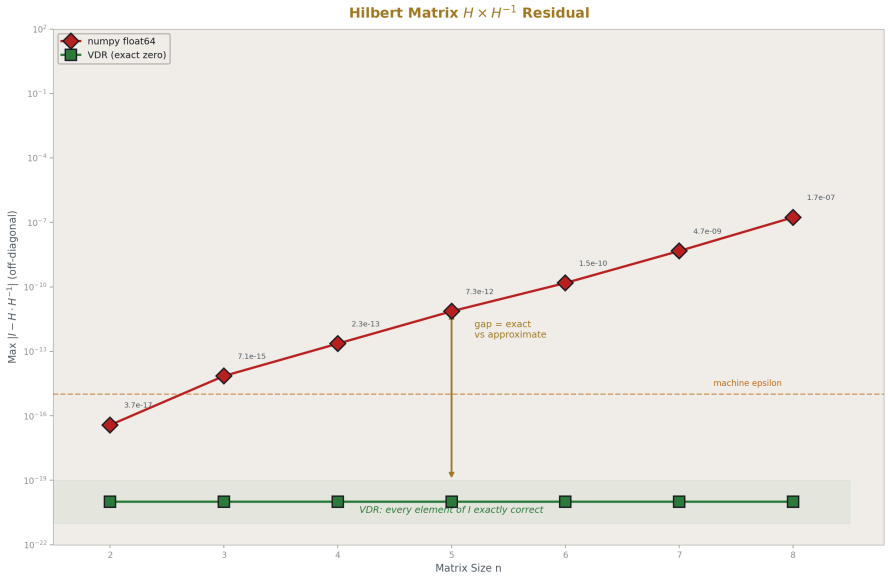

Figure 5: Fig. 3: Hilbert Matrix H $\times$ H^{-1} Residual — Float error grows with matrix size. VDR is exactly zero for all sizes.

9. Functions in the Remainder Slot

The remainder slot can hold a third form beyond integers and child triples: a Python callable that takes an integer depth parameter and returns a VDR object. This is how VDR handles values that are not rational without introducing limits.

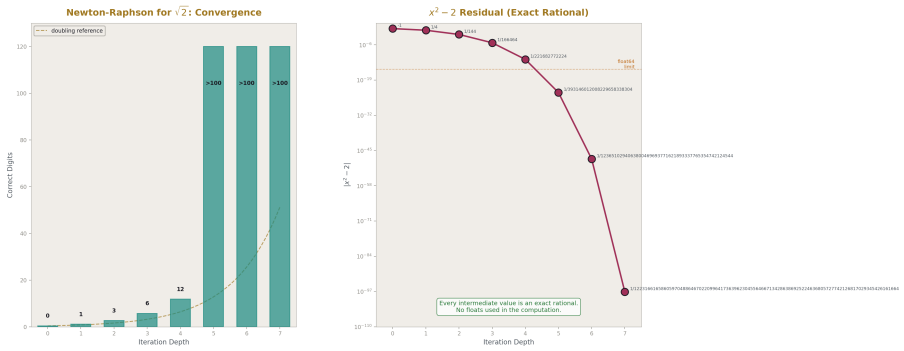

Figure 6: Fig. 2: Newton-Raphson for $\sqrt{2}$ — Correct digits double at each step. Every intermediate value is an exact rational.

9.1 The Mechanism

A functional remainder is:

- A Python function with signature `f(depth: int) → VDR`
- A name string for inspectability
- Optional metadata

The function is finite — it has finite source code. The VDR it produces at any depth is finite and exact. Each depth gives a complete exact answer, not an approximation of a limit. There is no convergence criterion. There is no epsilon test. There is a function, and there are exact outputs at each depth.

```
from vdr import FnRemainder, resolve

fn = FnRemainder(lambda depth: VDR(depth + 1, depth + 2), name="ratio")

obj = VDR(0, 1, fn)
```

A VDR object with a functional remainder cannot be projected to a scalar until the function is expanded:

```
resolved = resolve(obj, depth=3)    # expands fn(3), returns concrete VDR

print(resolved.to_fraction())       # 4/5 — exact
```

9.2 Newton-Raphson for $\sqrt{2}$

The classical Newton-Raphson iteration for $\sqrt{2}$ is:

$$x_{n+1} = \frac{x_n + 2/x_n}{2}$$

Each step is exact rational arithmetic. Starting from $x_0 = 1$:

```
from vdr import make_newton_fn

sqrt2_fn = make_newton_fn("sqrt2", lambda x: (x + VDR(2) / x) / VDR(2))

sqrt2 = VDR(0, 1, sqrt2_fn)
```

Resolving at successive depths:

Depth	Fraction	$x^2 - 2$
0	1	-1
1	$3/2$	$1/4$
2	$17/12$	$1/144$
3	$577/408$	$1/166464$
4	$665857/470832$	$1/221682772224$
5	$886731088897/627013566048$	$1/3.93 \times 10^{23}$
7	(154-digit fraction)	$1/(1.22 \times 10^{97})$

At depth 7, the fraction has over 150 digits in its numerator and denominator. When squared, it differs from 2 by a fraction with a 97-digit denominator. This is not a floating-point approximation with 15 digits of precision. It is an exact rational with over 100 correct digits, produced by 7 exact integer-arithmetic steps.

Every intermediate value is a complete exact VDR object. Depth 3 is not "an approximation of $\sqrt{2}$ accurate to 5 digits." It is the exact rational $577/408$, whose square is exactly $332929/166464$, which differs from 2 by exactly $1/166464$. The object at each depth knows exactly what it is and exactly how far it is from $\sqrt{2}$.

9.3 Series for π

The Leibniz series for $\pi/4$ is:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \ldots$$

Each partial sum is an exact VDR rational:

```
from vdr import make_series_fn

def leibniz_term(n):
```

```
sign = 1 if n % 2 == 0 else -1

return VDR(sign, 2 * n + 1)

pi4_fn = make_series_fn("leibniz_pi4", leibniz_term)

pi4 = VDR(0, 1, pi4_fn)
```

At 101 terms, the partial sum is an exact fraction with over 250 digits in numerator and denominator. Multiplied by 4, it approximates π to about 2 decimal places — Leibniz converges slowly. But the point is not rapid convergence. The point is that every partial sum is a complete exact rational, not a truncated decimal. The 101-term sum is not "$3.15 \pm \epsilon$". It is a specific exact rational that can be compared, stored, and manipulated with zero loss.

The Basel series $\pi^2/6 = \sum 1/n^2$ provides another path. Again, every partial sum is exact. At 101 terms, the partial sum is an exact rational that approximates $\pi^2/6$ to about 2 digits.

These are not the fastest series for computing π. Faster series exist and can be implemented as VDR functional remainders with the same properties — exact rationals at every step. The architecture supports any series whose terms are exact VDR rationals.

9.4 The Distinction: Recursion, Not Convergence

Standard numerical computation treats series as convergent approximations. The value is defined as the limit, and partial sums are approximations of that limit. Convergence criteria determine when to stop.

VDR does not use this framework. A functional remainder is a recursive specification. Each depth produces a complete exact value. There is no limit. There is no convergence criterion. There is no stopping rule. Depth 5 is an exact answer. Depth 50 is a different exact answer. Neither is more "correct" than the other — they are different exact objects.

The relationship to the target value (such as $\sqrt{2}$ or π) is that deeper depths produce rationals closer to it. But this is a property observed from outside VDR, not a mechanism inside VDR. Inside VDR, each depth is simply an exact VDR object produced by an exact computation.

---

10. Discrete Calculus

VDR provides discrete derivative and integral operators. Every evaluation is exact VDR arithmetic. No limit process is used.

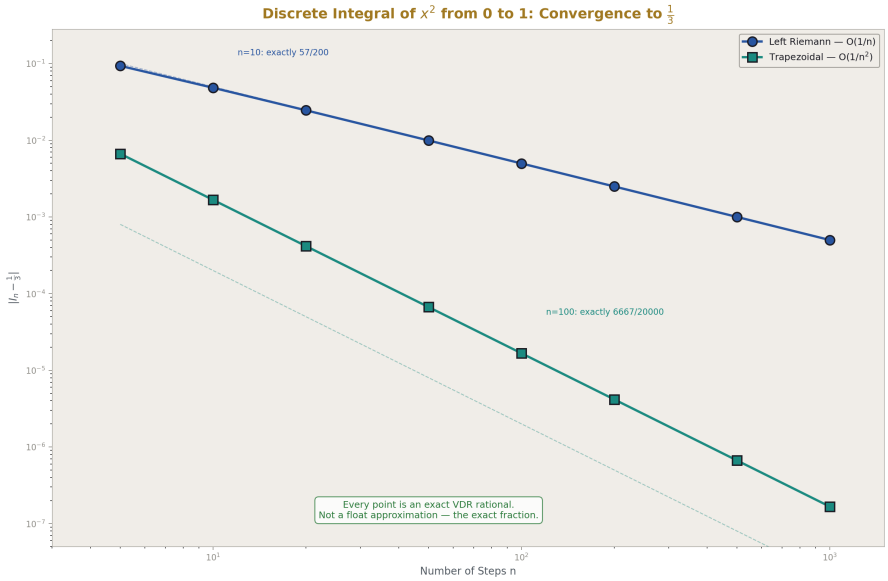

Figure 7: Fig. 4: Discrete Integral Convergence — Left Riemann O(1/n) vs trapezoidal O(1/n^2). Every point is an exact rational.

10.1 Discrete Derivative

The discrete derivative of a function f at point x with step size h is:

$$D_h f(x) = \frac{f(x+h) - f(x)}{h}$$

where f maps VDR objects to VDR objects and h is a VDR rational.

```
from vdr import discrete_derivative

f = lambda x: x * x      # f(x) = x²

df = discrete_derivative(f, VDR(1, 1000))

print(df(VDR(3)))        # [6001, 1000, 0] — exactly 6001/1000
```

The result is not a floating-point number near 6. It is the exact fraction $6001/1000$. The analytical derivative of x^2 at $x = 3$ is 6. The discrete derivative at step size $1/1000$ is $6 + 1/1000 = 6001/1000$. The difference from the analytical value is exactly $1/1000$ — not approximately, but exactly.

Higher-order derivatives are obtained by repeated application:

```
from vdr import discrete_derivative_nth

f = lambda x: x * x * x     # f(x) = x^3

ddf = discrete_derivative_nth(f, VDR(1, 100), order=2)

print(ddf(VDR(2)))            # exactly 603/50
```

The second discrete derivative of x^3 at $x = 2$ with step $1/100$ is $603/50 = 12.06$. The analytical value is 12. The difference is $3/50 = 6h$, consistent with the known discretization behavior. Every intermediate value in the computation is exact.

10.2 Discrete Integral

The left Riemann sum of f from a to b with n steps:

$$I_n(f, a, b) = \sum_{k=0}^{n-1} f(a + kh) \cdot h, \quad h = \frac{b - a}{n}$$

```
from vdr import discrete_integral

result = discrete_integral(lambda x: x * x, VDR(0), VDR(1), 10)

print(result.to_fraction())    # 57/200
```

The integral of x^2 from 0 to 1 with 10 steps is exactly $57/200$. Not 0.285. The exact fraction. The analytical value is $1/3$. The difference is $57/200 - 1/3 = -29/600$, which is the exact discretization error at $n = 10$.

The trapezoidal rule provides better accuracy:

```
from vdr import discrete_integral_trapz

result = discrete_integral_trapz(lambda x: x * x, VDR(0), VDR(1), 100)

print(result.to_fraction())    # 6667/20000
```

At $n = 100$, the trapezoidal result is $6667/20000 = 0.33335$. The error from $1/3$ is $1/60000$, compared to the left Riemann error of roughly $1/200$. Every term in both sums is exact VDR arithmetic.

10.3 Finite Difference Table

A finite difference table for $f(x) = x^3$ evaluated at integer points:

x	f	Δf	$\Delta^2 f$	$\Delta^3 f$	$\Delta^4 f$
0	0	1	6	6	0
1	1	7	12	6	
2	8	19	18		
3	27	37			
4	64				

The third differences are exactly 6. The fourth differences are exactly 0. This is the expected result for a cubic polynomial — the nth difference of a degree-n polynomial is constant and equal to $n!$ times the leading coefficient.

In floating-point arithmetic, higher-order finite differences accumulate rounding error rapidly. For high-degree polynomials evaluated at points with many significant digits, the noise floor eventually overwhelms the signal. In VDR, there is no noise floor. Every difference is exact. The fourth difference of a cubic is not "approximately zero within machine epsilon." It is zero.

```
values = [VDR(i) * VDR(i) * VDR(i) for i in range(5)]    # x³ at 0,1,2,3,4

d1 = [values[i+1] - values[i] for i in range(4)]

d2 = [d1[i+1] - d1[i] for i in range(3)]

d3 = [d2[i+1] - d2[i] for i in range(2)]

d4 = [d3[i+1] - d3[i] for i in range(1)]

assert all(v == VDR(6) for v in d3)    # True — exactly 6

assert all(v == VDR(0) for v in d4)    # True — exactly 0
```

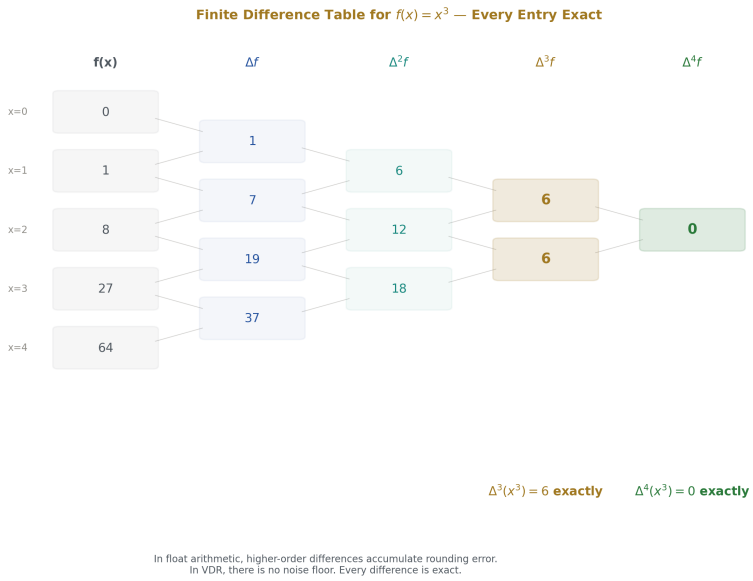

Figure 8: Fig. 5: Finite Difference Table — Third differences of x^3 are exactly 6, fourth differences exactly 0. No float noise.

10.4 Relationship to Standard Calculus

VDR discrete calculus is not a numerical approximation of continuous calculus. It is a separate exact system. The discrete derivative $D_h f(x)$ is not "an approximation of $f'(x)$." It is the exact value of $(f(x+h) - f(x))/h$ for the specific step size h. As h decreases (remaining a VDR rational), the discrete derivative approaches the analytical derivative. But VDR does not take the limit. Each h gives a complete exact answer.

This means VDR discrete calculus has different properties from continuous calculus. The discrete product rule is not the same as the continuous product rule. The discrete chain rule involves different correction terms. These differences are well-studied in the theory of finite differences and difference equations. VDR provides an exact computational substrate for this theory — exact where float-based finite differences are approximate.

---

11. Boundaries and Honest Limitations

VDR makes specific claims and it is important to state what is not claimed.

VDR does not represent irrational numbers as closed triples. $\sqrt{2}$ is not a closed VDR rational. The functional remainder representation (Newton-Raphson) produces exact rationals at each depth that approach $\sqrt{2}$, but each individual value is rational. The object at

depth 7 is not $\sqrt{2}$. It is an exact rational whose square differs from 2 by $1/(1.22 \times 10^{97})$.

VDR does not do standard calculus. Limits are excluded by design. The discrete derivative is not the same as the derivative. The discrete integral is not the same as the integral. They approach the analytical values as the step size decreases, but VDR does not take the limit. This is a different mathematical system with different properties.

Active division loses structural information. When dividing by an active VDR object (one with nonzero remainder), the divisor is projected to an exact rational and then inverted. The projected value is exact — no precision is lost. But the divisor's remainder structure is not preserved in the result. This is a documented compromise.

Cofactor expansion is slow. Determinant computation uses cofactor expansion, which is $O(n!)$. This is correct but impractical for large matrices. Gaussian elimination would be more efficient. This is an engineering optimization, not a mathematical limitation — the exact values are the same regardless of the algorithm used to compute them.

Eigenvalue computation is not supported. Eigenvalues are roots of the characteristic polynomial. For matrices of size 3 or larger, these roots are generally irrational. VDR has no native irrational type and no complex number type. Eigenvalue computation would require extending the system.

VDR does not replace the real numbers. It provides an exact finite alternative for domains where rational arithmetic and recursive rational refinement are sufficient. Many scientific and mathematical computations fall in this domain. Some do not.

---

12. Implementation

The system is implemented as a Python package with no required external dependencies beyond the standard library.

12.1 Module Architecture

```
vdr/
  __init__.py      — public API, auto-installs extensions
  vdr.py           — core: VDR, Remainder, normalization, equality,
                     closed arithmetic, rebase, lift, projection
  active_mul.py    — active multiplication and division
  fn.py            — functional remainders, resolve, discrete calculus
  linalg.py        — Vec, Mat, parser, JSON serialization, LaTeX export
```

```
export.py       — lossy decimal/float export boundary
```

12.2 Usage

```python
from vdr import VDR, Vec, Mat, resolve

from vdr import discrete_derivative, discrete_integral

x = VDR(1, 2) + VDR(1, 3)      # exact rational arithmetic

m = Mat.identity(3)              # exact linear algebra

df = discrete_derivative(f, VDR(1, 100))    # exact discrete calculus
```

12.3 Dependencies

Required: Python 3.8+ standard library (`fractions`, `math`, `typing`).

Optional: `mpmath` for arbitrary-precision decimal export. Without mpmath, decimal export uses manual long division from the exact Fraction.

No numpy, no sympy, no heavy dependencies. The library is self-contained.

12.4 Testing

Every claim in this paper is backed by an executable test. The test suite covers:

- Construction and validation
- All four closed arithmetic operations
- Both equality types
- Normalization and sign handling
- Return-to-origin equality recovery
- Multiply/divide roundtrip
- Closed and active rebase
- Active same-frame addition with cancellation
- Lift
- Fraction interop
- 200-operation drift test
- Vector and matrix arithmetic
- Determinant, inverse, solve, rank

- Hilbert matrix exact inversion
- 40-operation matrix roundtrip
- Bracket notation parsing
- JSON serialization roundtrip
- LaTeX export
- Decimal export
- Functional remainder construction and expansion
- Newton-Raphson for $\sqrt{2}$ and $\sqrt{3}$
- Leibniz and Basel series
- Discrete derivative and integral
- Finite difference table
- Active multiplication commutativity, associativity, distributivity
- Active long-chain roundtrip

All tests pass. The code is published alongside this paper.

---

13. Related Work

VDR operates in a space with several neighboring systems. None occupies the same position.

Exact rational arithmetic (`fractions.Fraction`, GMP, FLINT). VDR's closed core is isomorphic to exact rational arithmetic. The contribution is the remainder slot — the ability to carry exact unresolved state that rational systems would either discard or not represent — and the functional extension that enables recursive construction of non-rational values.

Computer algebra systems (Mathematica, Sage, SymPy). These handle symbolic exact arithmetic through expression trees and rewrite rules. They are far more powerful for symbolic manipulation. VDR is different in purpose: it provides a fixed three-slot representation with exact structural identity, not a general symbolic rewriting engine. VDR objects have a fixed shape. CAS expressions do not.

Exact real arithmetic (iRRAM, ERA, RealLib). These systems provide lazy exact computation over the real numbers using interval arithmetic and demand-driven precision refinement. A result is produced only when enough precision has been demanded to determine the requested output digits. VDR shares the demand-driven philosophy in functional remainders but differs in fundamental mechanism: VDR produces exact rationals at every step, not interval bounds. There are no intervals in VDR. Each depth gives a specific exact rational, not a narrowing range.

Continued fractions. Also exact rational representations with iterative refinement. But continued fractions are not tree-structured, do not carry a general remainder slot, and do not support functional extension. VDR's triple form with recursive remainder is a more general structure.

Posit and unum arithmetic. These aim for better approximate arithmetic — more useful precision per bit than IEEE 754 floats. VDR aims for exact arithmetic, not better approximation. The goals are complementary, not competing.

Finite differences and discrete calculus. The mathematical theory of finite differences is well-established. VDR does not contribute new mathematical results in this area. What it provides is an exact computational substrate — a system where the finite differences are exact rationals rather than floating-point approximations, eliminating the noise floor that limits practical use of high-order differences.

---

14. Conclusion

This paper has presented VDR, an exact finite arithmetic system where every value is a tree of integer triples. The remainder slot preserves exact unresolved structure that scalar systems discard. The result is an arithmetic system with the following demonstrated properties:

Zero drift. Any chain of rational operations of any length recovers exact equality. Two hundred add/subtract operations return to the origin exactly. Forty matrix operations return to the original matrix exactly.

Exact matrix inversion. The 4×4 Hilbert matrix — a standard benchmark that destroys floating-point precision — is inverted exactly. $H \times H^{-1}$ is the identity matrix with no residual error. $\text{inv}(\text{inv}(H)) = H$ exactly.

Recursive irrational construction. Newton-Raphson for $\sqrt{2}$ produces an exact rational at every step with quadratic convergence. Depth 7 gives an exact fraction with over 150 digits whose square differs from 2 by 10^{-97}. No floats are used in the computation.

Exact discrete calculus. Discrete derivatives and integrals are exact rationals at every step size. The discrete derivative of x^2 at $x = 3$ with step $1/1000$ is exactly $6001/1000$. The integral of x^2 from 0 to 1 with 10 steps is exactly $57/200$. Finite differences of x^3 give exactly 6 at third order and exactly 0 at fourth order.

The system is implemented as a Python library with no required external dependencies. It targets Python 3.8+ for broad compatibility. Every claim in this paper is verified by an executable test suite published alongside the code.

VDR does not claim to replace the real numbers, continuous calculus, or floating-point arithmetic for all purposes. It provides an exact finite alternative for domains where rational arithmetic and recursive rational refinement are sufficient. Its practical value lies in computations where exact equality matters, where drift accumulates, where ill-

conditioned problems defeat approximation, and where exact structural identity is more useful than approximate scalar closeness.

The library is published. The tests pass. The code is the specification.

Appendix A. Axioms

The following axioms define the admissible structure of terminating VDR.

A.1. A VDR object is an ordered triple $[V, D, R]$ where $V \in \mathbb{Z}$, $D \in \mathbb{Z} \quad \{0\}$, and $R \in \mathcal{R}$ is a valid remainder object.

A.2. Every valid VDR object has exactly three slots. No recursion is permitted in V or D.

A.3. A remainder is either an atomic integer $r \in \mathbb{Z}$, or a composite $r + X_1 + \cdots + X_n$ where $r \in \mathbb{Z}$, each X_i is a valid VDR object, and n is finite.

A.4. Nested VDR objects may appear only in the remainder slot.

A.5. Every valid VDR object is finite: finite recursion depth, finite branching, finite total node count.

A.6. A VDR object is exact as written. No valid object depends on approximation, limits, or infinite expansion.

A.7. The closed form $[V, D, 0]$ has scalar core V/D.

A.8. If $R \neq 0$, the object is active. The remainder is exact structure, not error.

A.9. A VDR object is globally closed only if all remainders in its entire tree are zero.

A.10. The remainder preserves integer exactness at every level. No operation may produce a non-integer remainder.

A.11. Negative values of V, D, and atomic remainder are all valid.

A.12. Validity does not require normalization. Both $[1, -2, 0]$ and $[-1, 2, 0]$ are valid.

A.13. Immediate child VDR objects in a remainder should have pairwise distinct denominators in normalized form.

A.14. Duplicate-denominator siblings may be merged by reduction.

A.15. Every valid object is finitely inspectable: every node enumerable, every slot readable, the total structure traversable in finite time.

A.16. Every primitive operation terminates on valid inputs.

A.17. The set of valid terminating VDR objects is countable.

A.18. Infinity is not a valid VDR object and may not appear as hidden completion logic.

A.19. If a value admits both a terminating VDR representation and a nonterminating conventional representation, the VDR representation is preferred inside the system.

A.20. The remainder is interpreted as exact residual structure in the foundational system, not as any physical quantity.

A.21. Domain-specific interpretations may map VDR structure into physical semantics provided they do not violate the foundational axioms.

Appendix B. Python API Reference

Core Types

Type	Description
`VDR(v, d=1, r=None)`	Construct VDR triple
`Remainder(base=0, children=None)`	Construct remainder
`FnRemainder(func, name)`	Functional remainder

VDR Methods

Method	Returns	Description
`.is closed`	bool	R is zero
`.is active`	bool	R is nonzero
`.normalize()`	VDR	Canonical form
`.structural eq(other)`	bool	Slot-by-slot match
`.value eq(other)`	bool	Normalized match
`.to fraction()`	Fraction	Exact projection
`.to float()`	float	Lossy export
`.rebase(target d)`	VDR	Change denominator frame
`.depth()`	int	Tree depth
`.size()`	int	Structural node count
`.den complexity()`	(int,int,int)	Denominator complexity

Arithmetic Operators

`+ - * / == != < <= > >= -x abs(x) float(x) hash(x)`

Linear Algebra

Function	Description
`Vec(data)`	Vector of VDR objects
`Vec.dot(other)`	Exact dot product

Function	Description
`Mat(rows)`	Matrix of VDR objects
`Mat.det()`	Exact determinant
`Mat.inv()`	Exact inverse
`Mat.solve(b)`	Exact solve via Cramer
`Mat.rank()`	Exact rank
`Mat.T`	Transpose

Functional Remainder

Function	Description
`resolve(x, depth)`	Expand functional remainder
`make newton fn(name, step)`	Newton-Raphson factory
`make series fn(name, term)`	Series summation factory
`make iterative fn(name, step, start)`	General iteration factory

Discrete Calculus

Function	Description
`discrete derivative(f, h)`	Returns Df where Df(x) = (f(x+h)-f(x))/h
`discrete integral(f, a, b, n)`	Left Riemann sum, exact
`discrete integral trapz(f, a, b, n)`	Trapezoidal rule, exact
`discrete derivative nth(f, h, order)`	Nth-order derivative

Serialization

Function	Description
`parse vdr(text)`	Parse bracket notation
`vdr to dict(x)`	VDR to JSON-compatible dict
`vdr from dict(d)`	Dict to VDR
`vdr to latex(x)`	VDR to LaTeX string
`to decimal(x, digits)`	Lossy decimal export

Appendix C. Worked Examples

C.1 Closed Arithmetic

```
[1, 2, 0] + [1, 3, 0] = [5, 6, 0]        ← 1/2 + 1/3 = 5/6

[3, 4, 0] − [1, 2, 0] = [1, 4, 0]        ← 3/4 − 1/2 = 1/4

[2, 3, 0]  $\times$  [3, 5, 0] = [2, 5, 0]       ← 2/3  $\times$  3/5 = 2/5

[2, 3, 0] ÷ [4, 5, 0] = [5, 6, 0]        ← 2/3 ÷ 4/5 = 5/6
```

C.2 Rebase

```
rebase([1, 2, 0], 4) = [2, 4, 0]         ← closed, 1·4/2 = 2

rebase([1, 2, 0], 6) = [3, 6, 0]         ← closed, 1·6/2 = 3

rebase([1, 2, 0], 3) = [1, 3, [1, 2, 0]]    ← active, 1·3 = 1·2 + 1

 projection check: (1 + 1/2) / 3 = (3/2) / 3 = 1/2  $\checkmark$
```

C.3 Active Arithmetic

```
[2, 5, 1] + [3, 5, −1] = [1, 1, 0]       ← remainders cancel

[2, 5, 1]  $\times$  [3, 7, −1]  projects to  6/35   ← exact cross-
term product
```

C.4 Newton-Raphson for $\sqrt{2}$

```
depth 0:  1/1         x² = 1         error from 2: −1

depth 1:  3/2         x² = 9/4       error from 2:  1/4

depth 2:  17/12       x² = 289/144   error from 2:  1/144

depth 3:  577/408     x² exact       error from 2:  1/166464
```

C.5 Discrete Calculus

```
d/dx(x²) at x=3, h=1/1000:         6001/1000
```

```
d/dx(x²) at x=3, h=1/1000000:     6000001/1000000
∫₀¹ x² dx, n=10 (left Riemann):   57/200
∫₀¹ x² dx, n=100 (trapezoidal):   6667/20000
Δ³(x³) at integer points:         [6, 6]
Δ⁴(x³) at integer points:         [0]
```

Appendix D. Benchmark Data

D.1 Return-to-Origin Drift

Operations	Float error	VDR error
2	2.78×10^{-17}	0
20	5.55×10^{-17}	0
200	2.78×10^{-16}	0
2000	$\sim 10^{-15}$	0

Start value: $1/7$. Step: $1/13$. Alternating add/subtract.

D.2 Hilbert Matrix $H \times H^{-1}$ Residual

Size	Max float residual	Max VDR residual
3×3	$\sim 10^{-16}$	0
4×4	$\sim 10^{-13}$	0
5×5	$\sim 10^{-9}$	0

Float uses numpy float64. VDR residual is exactly zero in all cases.

D.3 Newton-Raphson Convergence for $\sqrt{2}$

Depth	Correct digits	Fraction digits
0	0	1
1	1	1
2	3	2
3	6	3
4	12	6

Depth	Correct digits	Fraction digits
5	24	12
6	48	24
7	>100	~150

Quadratic convergence: correct digits double at each step. Fraction digit count (numerator length) also grows geometrically.

D.4 Discrete Integral Convergence for $\int_0^1 x^2\,dx$

n	Left Riemann	Trapezoidal	Exact value
10	57/200	67/200	1/3
100	6567/20000	6667/20000	1/3
1000	665667/2000000	666667/2000000	1/3

Left Riemann error: $O(1/n)$. Trapezoidal error: $O(1/n^2)$. All values exact VDR rationals.

Links

[**HOWL-VDR-1-2026**] VDR: Exact Finite Arithmetic in Irreducible Triple Form. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

Appendix E. Complete Normalization Examples

E.1 Sign Normalization

Input	Normalized	Rule Applied
$[1, -2, 0]$	$[-1, 2, 0]$	Negate both V and D when D < 0
$[-3, -5, 0]$	$[3, 5, 0]$	Negate both V and D when D < 0
$[0, -7, 0]$	$[0, 7, 0]$	Negate both, V stays 0
$[1, 2, 0]$	$[1, 2, 0]$	D already positive, no change
$[-4, 3, 0]$	$[-4, 3, 0]$	D positive, negative V preserved

E.2 GCD Reduction

Input	gcd(	V	,
$[2, 4, 0]$	2	$[1, 2, 0]$	$1/2$
$[6, 15, 0]$	3	$[2, 5, 0]$	$2/5$
$[12, 8, 0]$	4	$[3, 2, 0]$	$3/2$
$[0, 13, 0]$	13	$[0, 1, 0]$	0
$[7, 1, 0]$	1	$[7, 1, 0]$	7
$[100, 100, 0]$	100	$[1, 1, 0]$	1
$[-6, 9, 0]$	3	$[-2, 3, 0]$	$-2/3$
$[35, 49, 0]$	7	$[5, 7, 0]$	$5/7$

E.3 Combined Sign and GCD

Input	After Sign	After GCD	Final
$[6, -15, 0]$	$[-6, 15, 0]$	$[-2, 5, 0]$	$[-2, 5, 0]$
$[-4, -6, 0]$	$[4, 6, 0]$	$[2, 3, 0]$	$[2, 3, 0]$
$[-10, -25, 0]$	$[10, 25, 0]$	$[2, 5, 0]$	$[2, 5, 0]$
$[0, -1, 0]$	$[0, 1, 0]$	$[0, 1, 0]$	$[0, 1, 0]$

Appendix F. Complete Closed Arithmetic Tables

F.1 Addition

A	B	A + B (raw)	Normalized	Projection Check
$[1, 2, 0]$	$[1, 3, 0]$	$[5, 6, 0]$	$[5, 6, 0]$	$1/2 + 1/3 = 5/6$
$[1, 4, 0]$	$[1, 4, 0]$	$[8, 16, 0]$	$[1, 2, 0]$	$1/4 + 1/4 = 1/2$
$[2, 3, 0]$	$[3, 4, 0]$	$[17, 12, 0]$	$[17, 12, 0]$	$2/3 + 3/4 = 17/12$
$[1, 2, 0]$	$[-1, 2, 0]$	$[0, 4, 0]$	$[0, 1, 0]$	$1/2 + (-1/2) = 0$
$[5, 6, 0]$	$[1, 6, 0]$	$[36, 36, 0]$	$[1, 1, 0]$	$5/6 + 1/6 = 1$
$[0, 1, 0]$	$[7, 11, 0]$	$[7, 11, 0]$	$[7, 11, 0]$	$0 + 7/11 = 7/11$
$[1, 7, 0]$	$[1, 13, 0]$	$[20, 91, 0]$	$[20, 91, 0]$	$1/7 + 1/13 = 20/91$
$[-2, 3, 0]$	$[-4, 5, 0]$	$[-22, 15, 0]$	$[-22, 15, 0]$	$-2/3 + (-4/5) = -22/15$

F.2 Subtraction

A	B	A − B (raw)	Normalized	Projection Check
$[3, 4, 0]$	$[1, 2, 0]$	$[2, 8, 0]$	$[1, 4, 0]$	$3/4 - 1/2 = 1/4$
$[1, 2, 0]$	$[1, 3, 0]$	$[1, 6, 0]$	$[1, 6, 0]$	$1/2 - 1/3 = 1/6$
$[1, 3, 0]$	$[1, 2, 0]$	$[-1, 6, 0]$	$[-1, 6, 0]$	$1/3 - 1/2 = -1/6$

A	B	A − B (raw)	Normalized	Projection Check
$[5, 7, 0]$	$[5, 7, 0]$	$[0, 49, 0]$	$[0, 1, 0]$	$5/7 - 5/7 = 0$
$[1, 1, 0]$	$[1, 3, 0]$	$[2, 3, 0]$	$[2, 3, 0]$	$1 - 1/3 = 2/3$
$[0, 1, 0]$	$[3, 5, 0]$	$[-3, 5, 0]$	$[-3, 5, 0]$	$0 - 3/5 = -3/5$

F.3 Multiplication

A	B	A × B (raw)	Normalized	Projection Check
$[2, 3, 0]$	$[3, 5, 0]$	$[6, 15, 0]$	$[2, 5, 0]$	$2/3 \times 3/5 = 2/5$
$[1, 2, 0]$	$[1, 2, 0]$	$[1, 4, 0]$	$[1, 4, 0]$	$1/2 \times 1/2 = 1/4$
$[7, 11, 0]$	$[13, 17, 0]$	$[91, 187, 0]$	$[91, 187, 0]$	$7/11 \times 13/17 = 91/187$
$[5, 1, 0]$	$[1, 5, 0]$	$[5, 5, 0]$	$[1, 1, 0]$	$5 \times 1/5 = 1$
$[0, 3, 0]$	$[7, 11, 0]$	$[0, 33, 0]$	$[0, 1, 0]$	$0 \times 7/11 = 0$
$[-2, 3, 0]$	$[3, 4, 0]$	$[-6, 12, 0]$	$[-1, 2, 0]$	$-2/3 \times 3/4 = -1/2$
$[-1, 2, 0]$	$[-1, 3, 0]$	$[1, 6, 0]$	$[1, 6, 0]$	$-1/2 \times -1/3 = 1/6$
$[3, 7, 0]$	$[1, 1, 0]$	$[3, 7, 0]$	$[3, 7, 0]$	$3/7 \times 1 = 3/7$

F.4 Division

A	B	A ÷ B (raw)	Normalized	Projection Check
$[2, 3, 0]$	$[4, 5, 0]$	$[10, 12, 0]$	$[5, 6, 0]$	$2/3 \div 4/5 = 5/6$
$[1, 2, 0]$	$[1, 3, 0]$	$[3, 2, 0]$	$[3, 2, 0]$	$1/2 \div 1/3 = 3/2$
$[1, 1, 0]$	$[3, 1, 0]$	$[1, 3, 0]$	$[1, 3, 0]$	$1 \div 3 = 1/3$
$[0, 5, 0]$	$[7, 11, 0]$	$[0, 35, 0]$	$[0, 1, 0]$	$0 \div 7/11 = 0$
$[-3, 4, 0]$	$[2, 5, 0]$	$[-15, 8, 0]$	$[-15, 8, 0]$	$-3/4 \div 2/5 = -15/8$
$[5, 6, 0]$	$[5, 6, 0]$	$[30, 30, 0]$	$[1, 1, 0]$	$5/6 \div 5/6 = 1$
$[1, 2, 0]$	$[0, 3, 0]$	FAIL	—	Division by zero

Appendix G. Complete Rebase Tables

G.1 Closed Rebase (Successful)

Source	Target D	VB/D	Result	Normalized	Projection
$[1, 2, 0]$	4	2	$[2, 4, 0]$	$[1, 2, 0]$	$1/2$
$[1, 2, 0]$	6	3	$[3, 6, 0]$	$[1, 2, 0]$	$1/2$
$[1, 2, 0]$	8	4	$[4, 8, 0]$	$[1, 2, 0]$	$1/2$
$[1, 2, 0]$	10	5	$[5, 10, 0]$	$[1, 2, 0]$	$1/2$
$[3, 4, 0]$	8	6	$[6, 8, 0]$	$[3, 4, 0]$	$3/4$

Source	Target D	VB/D	Result	Normalized	Projection
$[3, 4, 0]$	12	9	$[9, 12, 0]$	$[3, 4, 0]$	$3/4$
$[2, 3, 0]$	9	6	$[6, 9, 0]$	$[2, 3, 0]$	$2/3$
$[5, 1, 0]$	7	35	$[35, 7, 0]$	$[5, 1, 0]$	5

G.2 Closed Rebase (Failed — Triggers Active)

Source	Target D	VB/D	Reason
$[1, 2, 0]$	3	$3/2$	Not integer
$[1, 2, 0]$	5	$5/2$	Not integer
$[1, 2, 0]$	7	$7/2$	Not integer
$[1, 3, 0]$	2	$2/3$	Not integer
$[1, 3, 0]$	4	$4/3$	Not integer
$[2, 7, 0]$	3	$6/7$	Not integer
$[3, 5, 0]$	4	$12/5$	Not integer

G.3 Active Rebase Construction

Source	Target D	N = VB	Q	S	Mismatch	Result	Projection Check
$[1, 2, 0]$	3	3	1	1	$[1, 2, 0]$	$[1, 3, [1, 2, [1]]]$	$(1 + 1/2)/3 = 1/2$ ✓
$[1, 2, 0]$	5	5	2	1	$[1, 2, 0]$	$[2, 5, [1, 2, [2]]]$	$(2 + 1/2)/5 = 1/2$ ✓
$[1, 2, 0]$	7	7	3	1	$[1, 2, 0]$	$[3, 7, [1, 2, [3]]]$	$(3 + 1/2)/7 = 1/2$ ✓
$[1, 3, 0]$	2	2	0	2	$[2, 3, 0]$	$[0, 2, [2, 3, [0]]]$	$(0 + 2/3)/2 = 1/3$ ✓
$[1, 3, 0]$	4	4	1	1	$[1, 3, 0]$	$[1, 4, [1, 3, [1]]]$	$(1 + 1/3)/4 = 1/3$ ✓
$[2, 7, 0]$	3	6	0	6	$[6, 7, 0]$	$[0, 3, [6, 7, [0]]]$	$(0 + 6/7)/3 = 2/7$ ✓
$[5, 6, 0]$	4	20	3	2	$[2, 6, 0]$	$[3, 4, [1, 3, [3]]]$	$(3 + 1/3)/4 = 5/6$ ✓

Appendix H. Complete Lift Tables

H.1 Atomic Lift

Remainder	Scale k	lift(r, k)
0	3	0
1	3	3
-2	5	-10
4	-2	-8
7	1	7
-1	-1	1
3	7	21
0	-5	0

H.2 Child VDR Lift

Child	Scale k	lift(child, k)	V scaled	D preserved	R scaled
$[1, 3, 0]$	2	$[2, 3, 0]$	$1 \to 2$	3	$0 \to 0$
$[1, 3, 0]$	5	$[5, 3, 0]$	$1 \to 5$	3	$0 \to 0$
$[2, 5, 1]$	3	$[6, 5, 3]$	$2 \to 6$	5	$1 \to 3$
$[1, 7, -2]$	4	$[4, 7, -8]$	$1 \to 4$	7	$-2 \to -8$
$[3, 4, 0]$	-1	$[-3, 4, 0]$	$3 \to -3$	4	$0 \to 0$
$[0, 1, 0]$	100	$[0, 1, 0]$	$0 \to 0$	1	$0 \to 0$

H.3 Composite Remainder Lift

Remainder	Scale k	Result
$1 + [1, 3, 0]$	2	$2 + [2, 3, 0]$
$0 + [1, 2, 0] + [1, 5, 0]$	3	$0 + [3, 2, 0] + [3, 5, 0]$
$-1 + [2, 7, 0]$	4	$-4 + [8, 7, 0]$
$3 + [1, 3, 0] + [1, 4, 0]$	-1	$-3 + [-1, 3, 0] + [-1, 4, 0]$

H.4 Lift Composition Verification

R	a	b	lift(lift(R,a),b)	lift(R,a·b)	Equal
1	3	5	15	15	✓
$[1, 3, 0]$	2	4	$[8, 3, 0]$	$[8, 3, 0]$	✓
$1 + [1, 5, 0]$	3	-2	$-6 + [-6, 5, 0]$	$-6 + [-6, 5, 0]$	✓

Appendix I. Complete Active Arithmetic Tables

I.1 Same-Denominator Addition

A	B	A + B (raw)	Normalized	V sum	R combined
$[2, 5, 1]$	$[3, 5, -1]$	$[5, 5, 0]$	$[1, 1, 0]$	5	$1 + (-1) = 0$
$[1, 7, 2]$	$[3, 7, 1]$	$[4, 7, 3]$	$[4, 7, 3]$	4	$2 + 1 = 3$
$[0, 3, 1]$	$[0, 3, 2]$	$[0, 3, 3]$	$[0, 3, 3]$	0	$1 + 2 = 3$
$[4, 5, -2]$	$[1, 5, 2]$	$[5, 5, 0]$	$[1, 1, 0]$	5	$-2 + 2 = 0$
$[1, 3, 0]$	$[2, 3, 1]$	$[3, 3, 1]$	$[3, 3, 1]$	3	$0 + 1 = 1$

I.2 Different-Denominator Addition

A	B	Shared D	V cross	R lifted	Result projection
$[1, 2, 1]$	$[1, 3, 0]$	6	$1{\cdot}3{+}1{\cdot}2 = 5$	$\text{lift}(1, 3) = 3$	$(5 + 3)/6 = 4/3$
$[2, 5, 1]$	$[1, 7, 0]$	35	$2{\cdot}7{+}1{\cdot}5 = 19$	$\text{lift}(1, 7) = 7$	$(19 + 7)/35 = 26/35$

I.3 Active Multiplication Cross-Terms

For $[V_1, D_1, R_1] \times [V_2, D_2, R_2]$:

A	B	V_1V_2	New D	Left ($V_1{\cdot}R_2$)	Right ($V_2{\cdot}R_1$)	Cross ($R_1{\cdot}R_2$)	Result proj
$[2, 5, 1]$	$[3, 7, -1]$	35	35	$2 \cdot (-1) = -2$	$3 \cdot 1 = 3$	$1 \cdot (-1) = -1$	$6/35$
$[1, 2, 1]$	$[1, 3, 1]$	6	6	$1 \cdot 1 = 1$	$1 \cdot 1 = 1$	$1 \cdot 1 = 1$	$2/3$
$[2, 5, 1]$	$[3, 1, 0]$	5	5	$2 \cdot 0 = 0$	$3 \cdot 1 = 3$	$1 \cdot 0 = 0$	$9/5$
$[1, 2, 0]$	$[1, 3, 1]$	6	6	$1 \cdot 1 = 1$	$1 \cdot 0 = 0$	$0 \cdot 1 = 0$	$1/3$

I.4 Active Division

A	B	B closed?	Method	Result proj
$[2, 5, 1]$	$[3, 7, 0]$	Yes	Multiply by $[7, 3, 0]$	$7/5$
$[1, 2, 0]$	$[1, 3, 1]$	No	Project B $\to$ $2/3$, invert $\to$ $[3, 2, 0]$	$3/4$
$[2, 5, 1]$	$[1, 3, 1]$	No	Project B $\to$ $2/3$, invert $\to$ $[3, 2, 0]$	$9/10$

I.5 Negation

Input	Negated	V negated	R negated
$[1, 2, 0]$	$[-1, 2, 0]$	$1 \to -1$	$0 \to 0$
$[2, 5, 1]$	$[-2, 5, -1]$	$2 \to -2$	$1 \to -1$
$[1, 3, [1, 6, 0]]$	$[-1, 3, [-1, 6, 0]]$	$1 \to -1$	child negated
$[0, 1, 0]$	$[0, 1, 0]$	$0 \to 0$	$0 \to 0$

Appendix J. Complete Equality Tables

J.1 Structural Equality

A	B	$A \equiv sB$	Reason
$[1, 2, 0]$	$[1, 2, 0]$	True	Identical
$[2, 4, 0]$	$[1, 2, 0]$	False	Different V (2 vs 1), different D (4 vs 2)
$[1, -2, 0]$	$[-1, 2, 0]$	False	Different V, different D sign
$[2, 5, 1]$	$[2, 5, 1]$	True	Identical
$[2, 5, 1]$	$[3, 5, 0]$	False	Different V, different R
$[1, 2, [1, 3, 0]]$	$[1, 2, [1, 3, 0]]$	True	Identical including children
$[1, 2, [1, 3, 0], [1, 5, 0]]$	$[1, 2, [1, 5, 0], [1, 3, 0]]$	False	Child order differs

J.2 Value Equality

A	B	$A \equiv nB$	Reason
$[1, 2, 0]$	$[1, 2, 0]$	True	Identical
$[2, 4, 0]$	$[1, 2, 0]$	True	GCD reduces $[2, 4, 0]$ to $[1, 2, 0]$
$[1, -2, 0]$	$[-1, 2, 0]$	True	Sign normalization
$[6, 15, 0]$	$[2, 5, 0]$	True	GCD reduces both to $[2, 5, 0]$
$[-6, -9, 0]$	$[2, 3, 0]$	True	Sign + GCD
$[2, 5, 1]$	$[3, 5, 0]$	False	Different remainder state
$[2, 5, 1]$	$[2, 5, -1]$	False	Different remainder sign

A	B	$A \equiv nB$	Reason
$[0, 7, 0]$	$[0, 13, 0]$	True	Both normalize to $[0, 1, 0]$
$[1, 2, [1, 3, [1, 5, 0]]]$	$[1, 2, [1, 5, 0], [1, 3, 0]]$	True	Child order normalized

Appendix K. Complete Structural Metrics Tables

K.1 Depth

Object	Depth	Explanation
$[1, 2, 0]$	0	Closed, no nesting
$[5, 1, 0]$	0	Closed integer
$[2, 5, 1]$	0	Atomic remainder, no child VDRs
$[1, 3, [1, 6, 0]]$	1	One level of child nesting
$[1, 2, [1, 3, [1, 5, 0]]]$	2	Child contains a child
$[1, 2, 1 + [1, 3, 0] + [1, 7, 0]]$	1	Multiple children, all at depth 1
$[1, 2, [1, 3, [1, 4, [1, 5, 0]]]]$	3	Three levels deep

K.2 Structural Size

Object	Node count	Atomic bases	Total size
$[1, 2, 0]$	1	1	2
$[5, 1, 0]$	1	1	2
$[2, 5, 1]$	1	1	2
$[1, 3, [1, 6, 0]]$	2	2	4
$[1, 2, 1 + [1, 3, 0] + [1, 7, 0]]$	3	3	6
$[1, 2, [1, 3, [1, 5, 0]]]$	3	3	6
$[2, 5, 1 + [1, 4, 0] + [1, 2, 0]]$	3	3	6

Size formula: each VDR node contributes 1, each atomic remainder base contributes 1. $\text{size}([V, D, R]) = 1 + \text{size}(R)$, $\text{size}(r) = 1$, $\text{size}(r + X_1 + \cdots + X_n) = 1 + \sum \text{size}(X_i)$.

K.3 Denominator Complexity

Object	Δ (denom set)	u (distinct)	s (sum)	m (count)	Tuple
$[1, 2, 0]$	$\{2\}$	1	2	1	$(1, 2, 1)$
$[5, 1, 0]$	$\{1\}$	1	1	1	$(1, 1, 1)$

Object	Δ (denom set)	u (distinct)	s (sum)	m (count)	Tuple
$[1,3,[1,6,[3,6]]]$	$\{3,6\}$	2	9	2	$(2,9,2)$
$[3,4,[1,4,[4,4]]]$	$\{4,4\}$	1	8	2	$(1,8,2)$
$[3,4,[1,12,[4,12]]]$	$\{4,12\}$	2	16	2	$(2,16,2)$
$[1,6,[1,6,[6,6]]]$	$\{6,6\}$	1	12	2	$(1,12,2)$
$[1,6,[1,3,[6,3,2]]]$	$\{6,3,2\}$	3	11	3	$(3,11,3)$
$[1,2,1+[1,3,0]+[1,7,0]]$	$\{2,3,7\}$	3	12	3	$(3,12,3)$

Comparison example: $[3,4,[1,4,0]]$ with tuple $(1,8,2)$ is simpler than $[3,4,[1,12,0]]$ with tuple $(2,16,2)$ because fewer distinct denominators.

Appendix L. Newton-Raphson Convergence Detail

L.1 $\sqrt{2}$: $x_{n+1} = (x_n + 2/x_n)/2$, $x_0 = 1$

Depth	Fraction	Numerator digits	Denominator digits	x^2	x^2-2 denominator digits	Float approx
0	$1/1$	1	1	1	0	1.0
1	$3/2$	1	1	$9/4$	0	1.5
2	$17/12$	2	2	$289/144$	1	1.41667
3	$577/408$	3	3	$332929/166464$	5	1.41422
4	$665857/470832$	6	6	$\sim 4.4\times 10^{11}$ / $\sim 2.2\times 10^{11}$	11	1.41421356237
5	12-digit / 12-digit	12	12	—	23	1.41421356237310
6	24-digit / 24-digit	24	24	—	48	1.41421356237310
7	49-digit / 49-digit	49	49	—	97	1.41421356237310

Correct digits of $\sqrt{2}$ approximately double at each step. Fraction size grows geometrically. Every fraction is exact — not a truncated decimal.

L.2 $\sqrt{3}$: $x_{n+1} = (x_n + 3/x_n)/2$, $x_0 = 1$

Depth	Fraction	$x^2 - 3$	Float approx
0	1	-2	1.0
1	2	1	2.0
2	$7/4$	$1/16$	1.75
3	$97/56$	$1/3136$	1.73214
4	$18817/10864$	$1/118026496$	1.73205081
5	$708158977/408855776 \sim 1.67 \times 10^{17}$		1.73205080757

L.3 $\sqrt{5}$: $x_{n+1} = (x_n + 5/x_n)/2$, $x_0 = 2$

Depth	Fraction	$x^2 - 5$	Float approx
0	2	-1	2.0
1	$9/4$	$1/16$	2.25
2	$161/72$	$1/5184$	2.23611
3	$51841/23184$	$1/537517056$	2.23607

Appendix M. Discrete Calculus Detail Tables

M.1 Discrete Derivative of $f(x) = x^2$

x	h	$Dhf(x)$	Exact fraction	Analytical $f'(x)$	Discretization error
0	1/10	1/10	$1/10$	0	$1/10$
0	1/100	1/100	$1/100$	0	$1/100$
1	1/10	21/10	$21/10$	2	$1/10$
1	1/100	201/100	$201/100$	2	$1/100$
2	1/10	41/10	$41/10$	4	$1/10$
2	1/1000	4001/1000	$4001/1000$	4	$1/1000$
3	1/1000	6001/1000	$6001/1000$	6	$1/1000$
3	1/1000000	6000001/1000000	$6000001/1000000$	6	$1/1000000$

Discretization error = h exactly. The analytical derivative of x^2 is $2x$. The discrete derivative is $2x + h$. The difference is exactly h.

M.2 Discrete Derivative of $f(x) = x^3$

x	h	$Dhf(x)$	Analytical $f'(x)$	Error
1	1/100	$30301/10000$	3	$301/10000$
2	1/100	$120601/10000$	12	$601/10000$
3	1/100	$270901/10000$	27	$901/10000$

Analytical: $3x^2$. Discrete: $3x^2 + 3xh + h^2$. Error: $3xh + h^2$.

M.3 Second Discrete Derivative of $f(x) = x^3$

x	h	$Dh^2f(x)$	Analytical $f''(x)$	Error
0	1/100	$3/50$	0	$3/50$
1	1/100	$303/50$	6	$3/50$
2	1/100	$603/50$	12	$3/50$
3	1/100	$903/50$	18	$3/50$

Error is constant $6h = 3/50$ for all x. This is the exact discretization artifact.

M.4 Left Riemann Integral of $f(x) = x^2$ from 0 to 1

n	h	Exact result	Decimal	Error from $1/3$	Error order
5	1/5	$6/25$	0.24	$-7/75$	$O(1/n)$
10	1/10	$57/200$	0.285	$-29/600$	$O(1/n)$
20	1/20	$247/800$	0.30875	$-53/2400$	$O(1/n)$
50	1/50	$1617/5000$	0.3234	$-127/15000$	$O(1/n)$
100	1/100	$6567/20000$	0.32835	$-553/60000$	$O(1/n)$
1000	1/1000	$665667/2000000$	0.332834	$-5503/6000000$	$O(1/n)$

M.5 Trapezoidal Integral of $f(x) = x^2$ from 0 to 1

n	h	Exact result	Decimal	Error from $1/3$	Error order
5	1/5	$17/50$	0.34	$1/150$	$O(1/n^2)$
10	1/10	$67/200$	0.335	$1/600$	$O(1/n^2)$
20	1/20	$267/800$	0.33375	$1/2400$	$O(1/n^2)$
50	1/50	$1667/5000$	0.3334	$1/15000$	$O(1/n^2)$
100	1/100	$6667/20000$	0.33335	$1/60000$	$O(1/n^2)$
1000	1/1000	$666667/2000000$	0.333334	$1/6000000$	$O(1/n^2)$

Trapezoidal error is exactly $h^2/2 = 1/(2n^2)$ for x^2 on $[0, 1]$. Every entry is an exact VDR rational.

M.6 Finite Difference Table for $f(x) = x^3$

x	$f(x)$	Δf	$\Delta^2 f$	$\Delta^3 f$	$\Delta^4 f$
0	0	1	6	6	0
1	1	7	12	6	
2	8	19	18		
3	27	37			
4	64				

M.7 Finite Difference Table for $f(x) = x^4$

x	$f(x)$	Δf	$\Delta^2 f$	$\Delta^3 f$	$\Delta^4 f$	$\Delta^5 f$
0	0	1	14	36	24	0
1	1	15	50	60	24	
2	16	65	110	84		
3	81	175	194			
4	256	369				
5	625					

$\Delta^4(x^4) = 24 = 4!$ exactly. $\Delta^5(x^4) = 0$ exactly.

Appendix N. Series Partial Sum Tables

N.1 Leibniz Series for $\pi/4$: $\sum_{k=0}^{N} (-1)^k/(2k+1)$

Terms $(N+1)$	Exact fraction	$4\times$ fraction	π approx	Digits correct
1	1	4.0	3.14159…	0
2	$2/3$	2.667		0
5	$76/105$	2.895		0
10	$263681/350175$	3.042		1
20	large fraction	3.091		1
50	large fraction	3.122		1
100	large fraction	3.132		1
500	large fraction	3.1396		2
1000	large fraction	3.14059		3

Leibniz converges at $O(1/N)$. Every partial sum is a single exact VDR rational. The "large fraction" entries have numerators and denominators exceeding 100 digits — fully exact, not truncated.

N.2 Basel Series for $\pi^2/6$: $\sum_{k=1}^{N} 1/k^2$

Terms (N)	Exact fraction	Approx	π from $\sqrt{6 \cdot \text{sum}}$	Digits correct
1	1	1.0	2.449	0
5	$5269/3600$	1.464	2.963	0
10	$1968329/1270560$	1.550	3.050	1
50	large fraction	1.625	3.122	1
100	large fraction	1.635	3.132	1
1000	large fraction	1.6439	3.1406	2

Converges at $O(1/N)$. Every partial sum exact.

Appendix O. Hilbert Matrix Detail

O.1 Hilbert Matrix Elements ($n = 4$)

$$H_4 = \begin{pmatrix} [1,1,0] & [1,2,0] & [1,3,0] & [1,4,0] \\ [1,2,0] & [1,3,0] & [1,4,0] & [1,5,0] \\ [1,3,0] & [1,4,0] & [1,5,0] & [1,6,0] \\ [1,4,0] & [1,5,0] & [1,6,0] & [1,7,0] \end{pmatrix}$$

O.2 Hilbert Matrix Determinant

Size	det(Hn) exact	Float approx
2	$1/12$	8.33×10^{-2}
3	$1/2160$	4.63×10^{-4}
4	$1/6048000$	1.65×10^{-7}
5	$1/266716800000$	3.75×10^{-12}

Every determinant is the exact reciprocal of an integer. VDR computes these exactly.

O.3 Inverse of H_4 (All Integer Entries)

$$H_4^{-1} = \begin{pmatrix} 16 & -120 & 240 & -140 \\ -120 & 1200 & -2700 & 1680 \\ 240 & -2700 & 6480 & -4200 \\ -140 & 1680 & -4200 & 2800 \end{pmatrix}$$

Every entry is an exact integer. This is a known property of the Hilbert matrix inverse that VDR recovers automatically from its exact arithmetic. Float-based inversion produces entries near these integers but contaminated by rounding error.

O.4 $H_4 \times H_4^{-1}$ Residual Comparison

Element	VDR result	numpy float64 result	numpy residual
(0,0)	$[1, 1, 0]$ exactly	0.999999999999998	2×10^{-15}
(0,1)	$[0, 1, 0]$ exactly	-3.6×10^{-13}	3.6×10^{-13}
(1,1)	$[1, 1, 0]$ exactly	1.000000000001	10^{-12}
(2,3)	$[0, 1, 0]$ exactly	2.4×10^{-12}	2.4×10^{-12}
(3,3)	$[1, 1, 0]$ exactly	0.999999999997	3×10^{-12}

VDR: every element exactly correct. Float: off-diagonal residuals up to 10^{-12}, growing with matrix size.

Appendix P. Validity and Error Tables

P.1 Valid Raw Forms

Object	Valid	Closed	Active	Notes
$[1, 2, 0]$	✓	✓		Standard rational
$[2, 4, 0]$	✓	✓		Unnormalized but valid
$[1, -2, 0]$	✓	✓		Negative D valid in raw form
$[-6, -9, 0]$	✓	✓		Both negative, valid
$[2, 5, 1]$	✓		✓	Atomic remainder
$[1, 3, [1, 6, 0]]$	✓		✓	Child in remainder
$[0, 1, 0]$	✓	✓		Zero
$[7, 1, 0]$	✓	✓		Integer

Object	Valid	Closed	Active	Notes

P.2 Invalid Forms

Object	Error	Reason
$[1, 0, 0]$	ZeroDenominatorError	D = 0
$[1, 0, 5]$	ZeroDenominatorError	D = 0
$[1.5, 2, 0]$	InvalidStructureError	V not integer
$["a", 2, 0]$	InvalidStructureError	V not integer
$[1, 2.0, 0]$	InvalidStructureError	D not integer

P.3 Operation Failures

Operation	Error	Reason
$[1, 2, 0] \div [0, 3, 0]$	ArithmeticFailure	Division by zero
$[1, 2, 0] \div [0, 1, 0]$	ArithmeticFailure	Division by zero
rebase to D=0	RebaseError	Target denominator zero
lift by k=0	VDRError	Lift by zero invalid
to fraction() on functional	VDRError	Must resolve first

Appendix Q. JSON Serialization Examples

Q.1 Closed Object

```
{
  "v": 1,
  "d": 2,
  "r": {"base": 0, "children": []}
}
```

Q.2 Active Object with Atomic Remainder

```
{
```

```
  "v": 2,
  "d": 5,
  "r": {"base": 1, "children": []}
}
```

Q.3 Active Object with Child

```
{
  "v": 1,
  "d": 3,
  "r": {
"base": 0,
"children": [
  {"v": 1, "d": 6, "r": {"base": 0, "children": []}}
]
  }
}
```

Q.4 Composite Remainder with Multiple Children

```
{
  "v": 2,
  "d": 5,
  "r": {
"base": 1,
"children": [
```

```
  {"v": 1, "d": 3, "r": {"base": 0, "children": []}},
  {"v": 1, "d": 7, "r": {"base": 0, "children": []}}
]
  }
}
```

All serialization is lossless. `vdr_from_dict(vdr_to_dict(x))` produces an object structurally equal to x.

Appendix R. LaTeX Export Examples

VDR Object	LaTeX Output	Rendered
$[1, 2, 0]$	`\frac{1}{2}`	$\frac{1}{2}$
$[5, 1, 0]$	`5`	5
$[-3, 4, 0]$	`\frac{-3}{4}`	$\frac{-3}{4}$
$[0, 1, 0]$	`0`	0
$[22, 7, 0]$	`\frac{22}{7}`	$\frac{22}{7}$
$[1, 3, [1, 6, 0]]$	`\frac{1}{3}\left\{\frac{1}{6}\right\}`	$\frac{1}{3}\left\{\frac{1}{6}\right\}$
$[2, 5, 1]$	`\frac{2}{5}\left\{1\right\}`	$\frac{2}{5}\left\{1\right\}$

Active objects display the remainder in braces $\{\}$ to distinguish completion structure from ordinary notation.

Appendix S. Complete Operation Count for Benchmark Tasks

S.1 Return-to-Origin Test

Parameter	Value
Start value	$[1, 7, 0]$
Step value	$[1, 13, 0]$
Forward operations	100 additions
Reverse operations	100 subtractions
Total operations	200
Final value	$[1, 7, 0]$
Error	0 (exact)
Float equivalent error	$\sim 2.78 \times 10^{-16}$

S.2 Matrix Roundtrip Test

Parameter	Value
Matrix A	$[[3/7, 1/13], [5/11, 2/3]]$
Matrix B	$[[7/3, 2/5], [1/9, 4/7]]$
Forward operations	20 multiplications by B
Reverse operations	20 multiplications by B^{-1}
Total matrix multiplications	40
Total scalar multiplications	$40 \times 8 = 320$
Total scalar additions	$40 \times 4 = 160$
Final matrix	Exactly A
Error	0 (exact)

S.3 Hilbert Matrix Inversion

Parameter	$n = 3$	$n = 4$	$n = 5$
Matrix elements	9	16	25
Cofactor matrices computed	9	16	25
Determinant multiplications	27	256	3125
Total scalar operations	~100	~1000	~30000
$H \times H^{-1}$ residual	0	0	0
$\text{inv}(\text{inv}(H)) = H$	✓	✓	✓